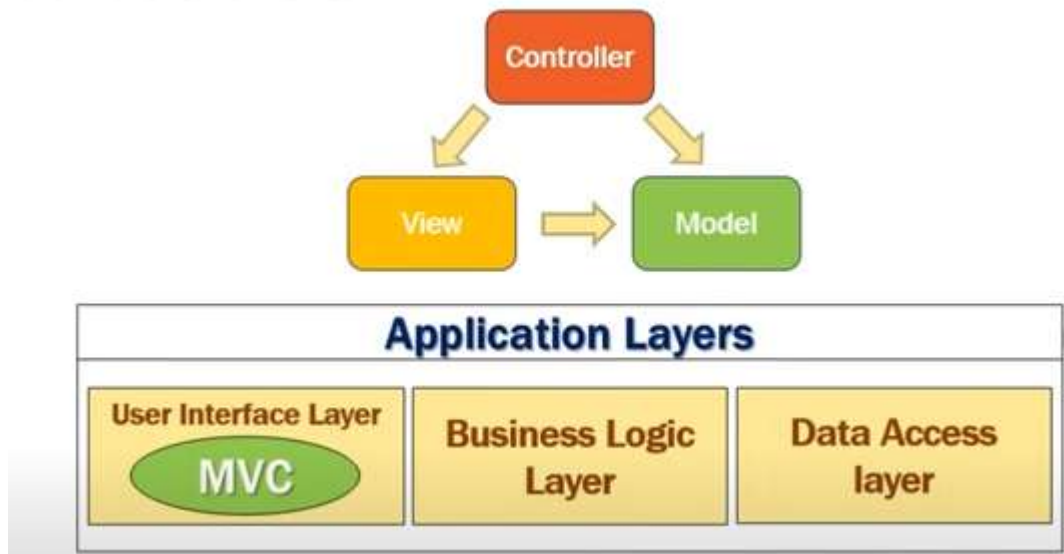


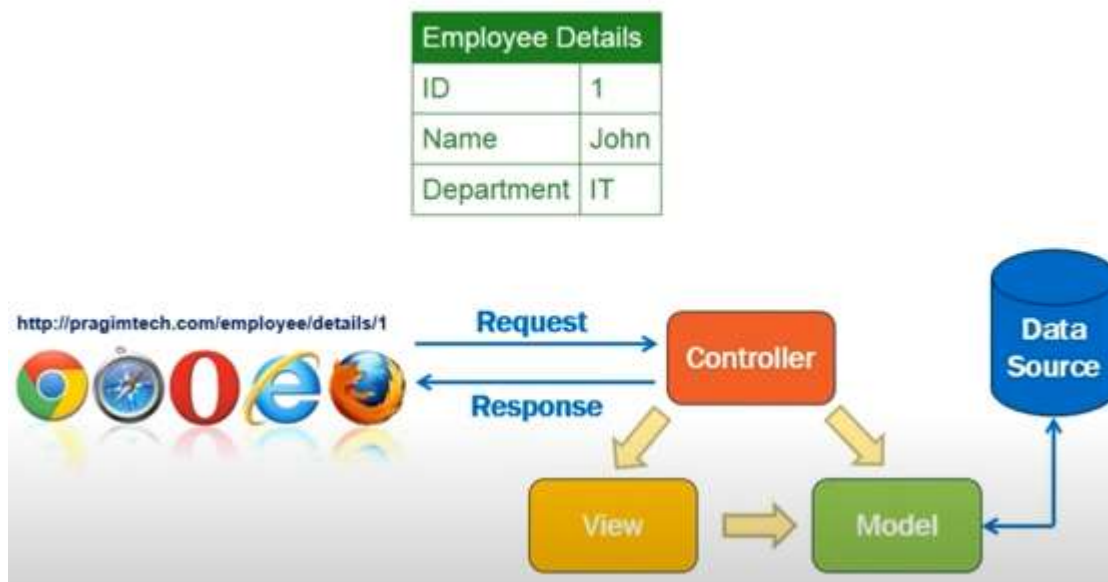
What is MVC



MVC = architectural design pattern for implementing the user interface layer of an application.

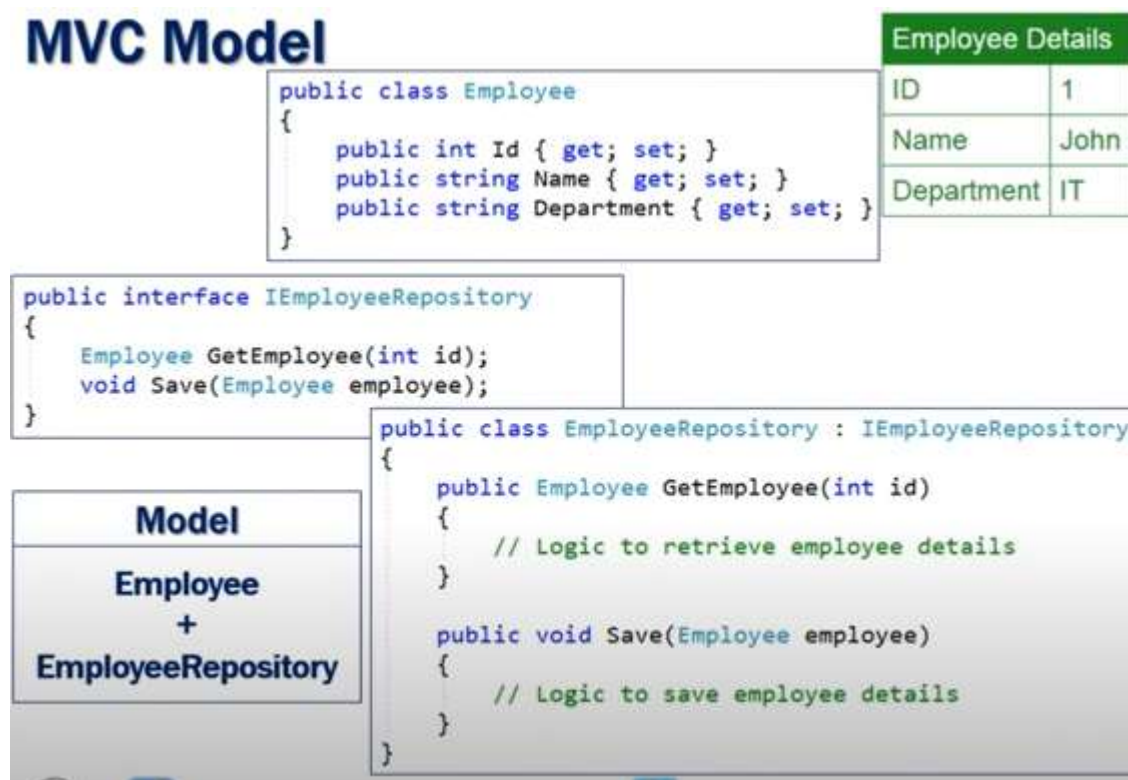
Application layers = User Interface (MVC) + Business Logic (Domain) + Data Access

How MVC Works



Логиката във View трябва да се сведе до минимум, колкото да покаже данните.

MVC Model



EmployeeRepository = Interface for EmployeeRepository

Може без интерфейс чисто технологично да се имплементира приложението, само с данни и обработка, но този допълнителен интерфейс ни позволява dependency injections (зависими вмъквания) за по-голяма гъвкавост, свобода и тестване.

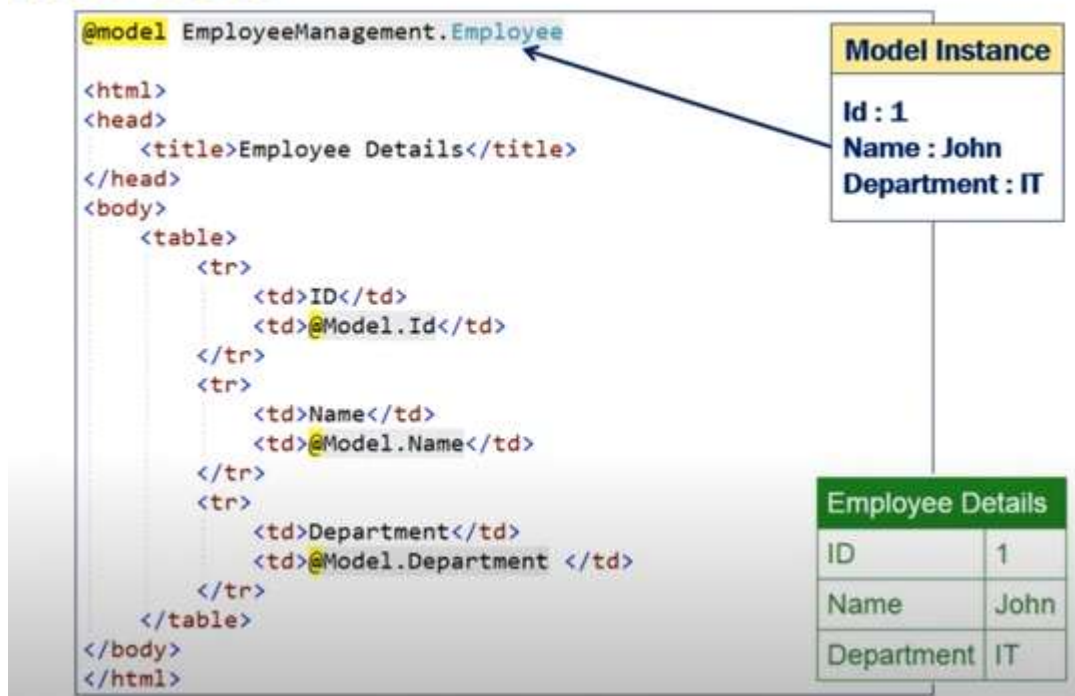
Т.е. ние, създавайки class Employee, все едно създаваме по-сложен ТИП данни – не прости типове като int и string, а по-сложен ТИП, който съдържа в себе си повече неща (int Id, string Name, string Department) и вече въз основа на него ще създаваме нови обекти от този клас (модел) Employee. Служител 1 employee1 ще бъде от ТИП като този клас, към който принадлежи Employee, т.е. извиквайки при създаването Employee employee1, ние ще знаем, че създаваме обект (като променлива) employee1, която ще съдържа в себе си Id, Name, Department.

После създаваме един Interface IEmployeeRepository, за да бъдат показвани нещата графично опростено на потребителите.

Накрая още един клас (модел) EmployeeRepository, който ще наследява (извиква и съдържа в себе си) интерфейса IEmployeeRepository.

View – трябва да е възможно най-опростен HTML template.

MVC View

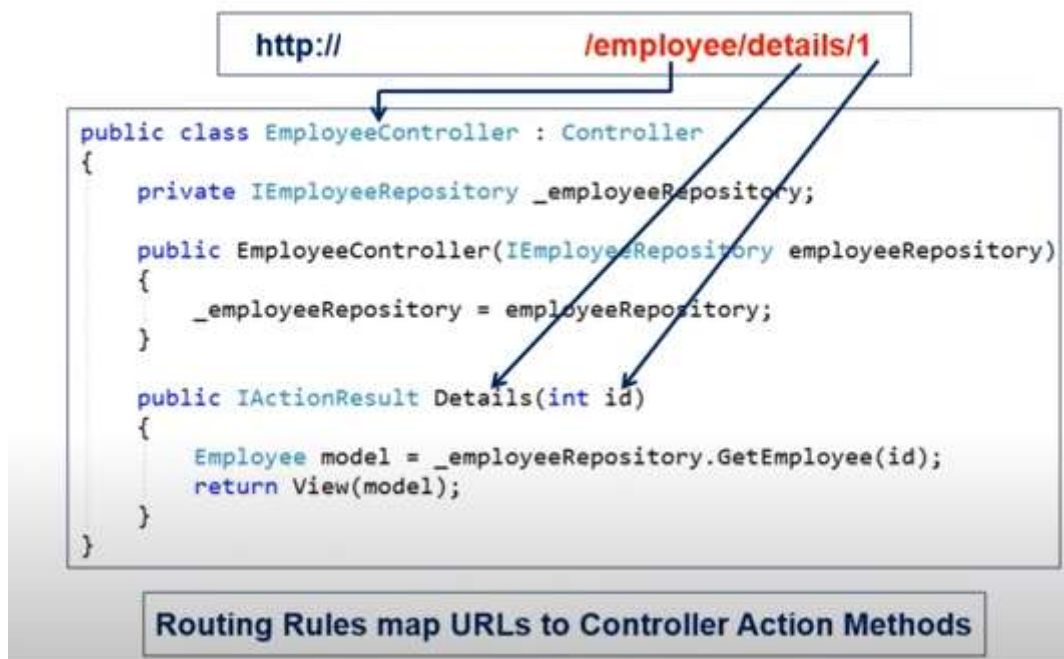


Посредством @Model. извикваме динамично стойностите на променливата (инстанция) от ТИП/клас /модел Employee. По този начин, чрез @ като част от C# код, не се изписват конкретните стойности, а те се намират, прихващат и показват като за всеки обект тип Employee те са различни.

Model -----> data -----> View (user) http -----> Controller

Адресирането в браузъра спазва конвенцията /{Controller}/{Action}/{id}

MVC Controller



* Routing rules map URLs to Controller Action Method

http request ----- Controller (builds the model) ----- response to user action

Summary

MVC is an architectural design pattern for implementing User Interface Layer of an application

- **Model** : Set of classes that represent data + the logic to manage that data
- **View** : Contains the display logic to present the Model data provided to it by the Controller
- **Controller** : Handles the http request, work with the model, and selects a view to render that model

MVC = архитектурен дизайн за интерфейс потребител-приложение

* Model = състои се задължително от 2 неща: класове за данни + логика за управлението им

* View = логика за дисплей на данните от модела чрез контролера

* Controller = обработва потребителските заявки чрез модела

Setup MVC in ASP.NET Core

➤ Step 1 : Add the MVC Services to the Dependency Injection Container

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddMvc();
}
```

➤ Step 2 : Add MVC middleware to the Request Pipeline

```
public void Configure(IApplicationBuilder app, IHostingEnvironment env)
{
    if (env.IsDevelopment())
    {
        app.UseDeveloperExceptionPage();
    }

    app.UseStaticFiles();
    app.UseMvcWithDefaultRoute();
}
```

- 1: services
- 2: pipeline

B Startup

```
1:     public void ConfigureServices(IServiceCollection services)
        {
            services.AddMvc();
        }

...
2:     app.UseStaticFiles();
        app.UseMvc();
        app.UseRouting();
```

DefaultRoute търси Index-секция в HomeController.cs, но понеже не го намира, при зададената последователност предава по pipeline към следващия middleware, който е terminal – WriteAsync(“Hello world!”). Т.е. ако не намери страница като index, която да покаже в браузъра „Hello from index.html!”, ще отиде към следващия слой и ще намери „Hello world!” и ще покаже него.

Важното е UseStaticFiles да бъде преди UseMvc, понеже статичните файлове (картинки и прочие) ще бъдат прихванати и обработени и следващия middleware UseMvc няма да се използва, а ще бъде превключен на къса верига, избягвайки излишна обработка.

От друга страна, при друга заявка, статичните файлове ще бъдат пропуснати до следващия middleware UseMvc, който ще ги обработи.


```

0 references
public void ConfigureServices(IServiceCollection services)
{
    services.AddMvc(option => option.EnableEndpointRouting = false);
}

// This method gets called by the runtime. Use this method to configure the HTTP request pipeline
0 references
public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
{
    if (env.IsDevelopment()) // bool, IsStaging, IsProduction, IsEnvironment("UAT")
    {
        app.UseDeveloperExceptionPage();
    }

    app.UseStaticFiles(); //Short-circle ordering
    app.UseMvc(rb => //app.UseMvcWithDefaultRouting(); error after .net 3.8
    {
        rb.MapRoute(
            name: "default",
            template: "{controller}/{action}/{id?}",
            defaults: new { controller = "Home", action = "Index" });
    });
    app.UseRouting();

    app.UseEndpoints(endpoints =>
    {
        endpoints.MapGet("/", async context =>
        {
            await context.Response.WriteAsync("Hello world!");
        });
    });
}

```

Template – шаблонът показва, че /контролер/действие е задължително да се укажат, а това, че са в къдрави скоби {} – че са с променливи стойности. Въпросителната id? показва, че може да има id, а може и да отсъства в адресирането, т.е. тя има незадължителен характер – не е необходимо да присъства във всички линкове. Например, за home/list принципно не би било необходимо за цял списък да се показва id, но за /home/edit/5 (или create, delete, update) трябва да видим кой точно елемент от списъка редактираме.

Т.е. казваме, че искаме да използваме тази конфигурационна услуга MVC и тя да бъде с приоритет пред EndpointRouting, който изписва „Hello world!“

```
services.AddMvc(option => option.EnableEndpointRouting = false);
```

.....

```
app.UseMvc(rb =>
```

```
{
```

```
    rb.MapRoute(
```

```
        name: "default",
```

```
        template: "{controller}/{action}/{id?}",
```

```
        defaults: new {controller = "Home", action = "Index"});
```

```
});
```

Условия за използване на MVC:

- 1) ConfigureServices -> `services.AddMvc();`
- 2) Configure pipeline middleware -> `app.UseMvc();`

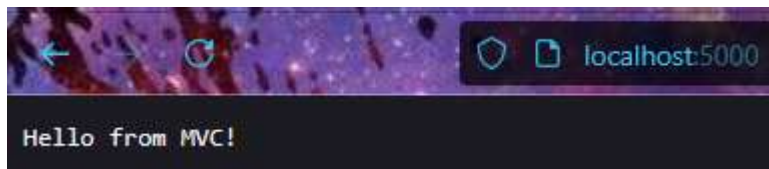
По този начин ще търси Index метод в HomeController. Нека го добавим в стандартна папка Controllers, нов клас HomeController.cs, метод Index



Във Visual Studio ще ни добави много код автоматично, но във Visual Code трябва да си го напишем сами:

```
namespace EmployeeManagement.Controllers
{
    public class HomeController
    {
        public string Index()
        {
            return "Hello from MVC!";
        }
    }
}
```

Резултат е „Hello from MVC!“, вместо „Hello from index.html“ или „Hello world!“



Това е така, т.к. UseMvc се намира преди терминалния middleware и когато неговото условие е изпълнено, той се стартира и другия след него няма вече шанс да се изпълни.

Т.е. в главния URL се изпълнява “Hello from MVC”, защото условията на extension- UseMvc()-middleware са изпълнени и като къса верига short-circle, след него WriteAsync(„Hello world!”) няма шанс да се изпълни.

Hello world се изпълнява при закоментиране на блока код преди него за UseMvc. Т.е. последователността има значение и по-предишния слой се изпълнява, а следващия не, т.е. това е къса верига, а не всички слоеве един след друг последователно да се изпълнят (дълга верига).

Проверявам с коментиране на блок код – в по-старите версии на Visual Studio/ Code използваме:

Ctrl + K (key) + C (comment) -> //закоментира всеки ред код

Ctrl + K (key) + U (uncomment) -> //премахва коментара на всеки ред код

Във 2022 използваме:

Ctrl + K -> закоментира //

Ctrl + / -> премахва коментар

Трябва да се има предвид, че при премахване на коментар автоматично, много пъти прави главните букви малки, които хвърля грешки, ако не сме наясно с тази уязвимост, затова, ако премахваме с клавишна комбинация, по-добре е това да бъде Ctrl + Z, ръчно изтриване на // и ако има грешки, да проверяваме за главните букви.

Mvc vs Mvc Core

services.
 Add.MvcCore(); - само ядро
 Add.Mvc(); - ядро + всичко

ASP.NET Core AddMvc vs AddMvcCore

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddMvc();
}

public void ConfigureServices(IServiceCollection services)
{
    services.AddMvcCore();
}
```

➤ **AddMvcCore()** method only adds the core MVC services

➤ **AddMvc()** method adds all the required MVC services

➤ **AddMvc()** method calls **AddMvcCore()** method internally

API = Application User Interface (подсказки)

Искаме да изучим повече функции за Mvc с включване на вграден клас Controller.

Всички типове резултати като ViewResult, JsonResult, PartialViewResult ... се предоставят от този вграден клас : Controller.

The screenshot shows the Visual Studio IDE with the `HomeController.cs` file open. The code is as follows:

```
1 namespace EmployeeManagement.Controllers
2
3 {
4     public class HomeController: Controller
5     {
6         public string Index()
7         {
8             return "Hello from MVC!";
9         }
10    }
11 }
12
13
```

A red arrow points to the `Controller` text in the class declaration on line 4, with the text "ще вкара библиотеката горе" (will load the library above). Another red arrow points to the `Controller` entry in the auto-completion dropdown, with the text "Enter". The dropdown list includes:

- `Controller` (Microsoft.AspNetCore.Mvc)
- `Controllers`
- `ControllerActionDescriptor` (Microsoft.AspNetCore.Mvc.Core)
- `ControllerActionEndpointConventionBuilder` (Microsoft.AspNetCore.Mvc.Core)
- `ControllerActivatorProvider` (Microsoft.AspNetCore.Mvc.Core)
- `ControllerAttribute` (Microsoft.AspNetCore.Mvc)
- `ControllerBase` (Microsoft.AspNetCore.Mvc)
- `ControllerBoundPropertyDescriptor` (Microsoft.AspNetCore.Mvc)
- `ControllerContext` (Microsoft.AspNetCore.Mvc)
- `ControllerContextAttribute` (Microsoft.AspNetCore.Mvc)

За него е необходима библиотеката с ред код `using Microsoft.AspNetCore.Mvc;`

Ако не успеем навреме да натиснем Enter при подсказката, другият начин за автокорекция е `Ctrl + .`

```
HomeController.cs 1 • Startup.cs 1
EmployeeManagement > Controllers > HomeController.cs > HomeController
1 namespace EmployeeManagement.Controllers
2
3 {
4     0 references
5     public class HomeController: Controller
6     {
7         0 references
8         public string Index()
9         {
10             return "Hello from MVC!";
11         }
12     }
13 }
```

Quick Fix

- using Microsoft.AspNetCore.Mvc;
- Microsoft.AspNetCore.Mvc.Controller
- Generate class 'Controller'
- Change 'Controller' to 'Controllers'.

Тя си отива на мястото най-отгоре:

```
HomeController.cs • Startup.cs 1
EmployeeManagement > Controllers > HomeController.cs > ...
1 using Microsoft.AspNetCore.Mvc;
2
3 namespace EmployeeManagement.Controllers
4
5 {
6     0 references
7     public class HomeController: Controller
8     {
9         0 references
10     public string Index()
11     {
12         return "Hello from MVC!";
13     }
14 }
15 }
```

MVC ()

прост тип

В момента имаме string -> return, което е прост тип данни. Но имаме и по-сложни.

Ще преименуваме Index да стане нещо друго като MVC, за да не се показва неговия action и ще демонстрираме по-сложен тип данни Json със съответния им JsonResult.

```
HomeController.cs X Startup.cs 1
EmployeeManagement > Controllers > HomeController.cs > HomeController > Index
1 using Microsoft.AspNetCore.Mvc;
2
3 namespace EmployeeManagement.Controllers
4
5 {
6     public class HomeController: Controller
7     {
8         public string MVC()
9         {
10             return "Hello from MVC!";
11         }
12
13         public JsonResult Index()
14         {
15             return Json(new {id=1, name="XII class!"}); //new anonymus object
16         }
17     }
18
19 }
```

необходима библиотека за разпознаване

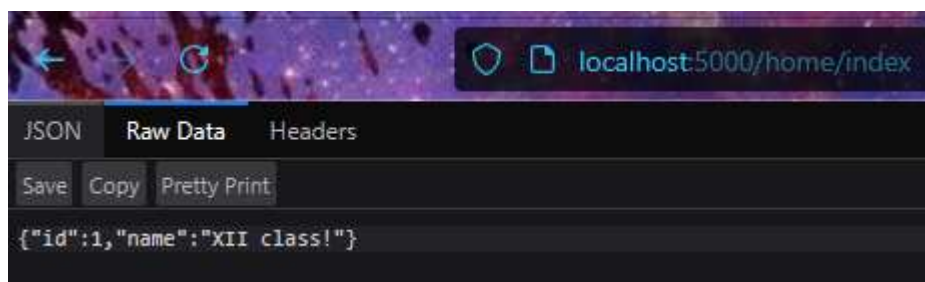
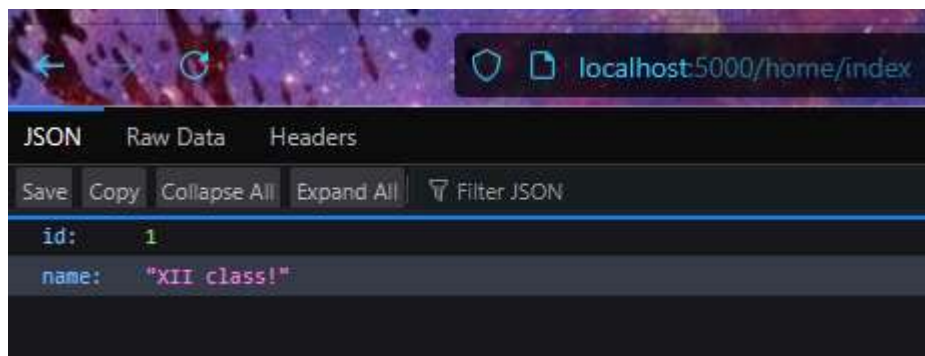
наследяване на вградени функции

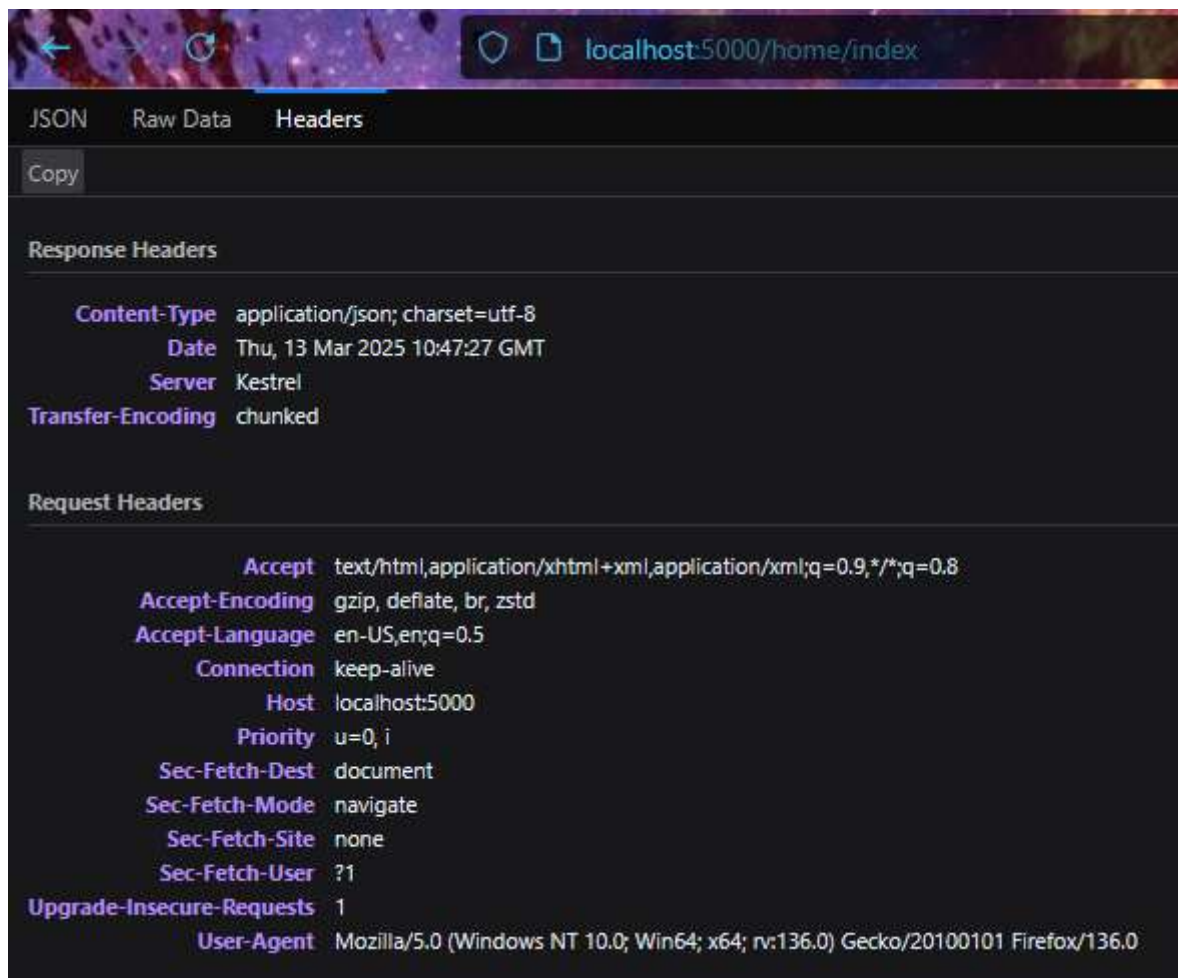
прост тип, преименувахме, за да не пречи

сложен тип

Вече можем да го намерим и извикаме от адрес /home/MVC

Резултатът в браузъра **/home/index** (т.е. нашия HomeController -> action Index):





Вижда се колко много данни се предоставят от този сложен тип json.

Може да се тества и последна настройка за MVC в Startup.cs:

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddMvcCore(option => option.EnableEndpointRouting = false);
}
```

Принципно AddMvcCore (ядро) е с по-малко възможности от просто AddMvc, но и по двата начина в новите версии работи. В по-старите версии нямаше да ни позволи с Core да използваме сложен тип json.

```

1 reference
private IConfiguration (extension) IMvcCoreBuilder
IServiceCollection.AddMvcCore(Action<Microsoft.AspNetCore.Mvc.MvcOptions> setupAction) (+ 1
0 references
public Startup overload)
{
    Adds the minimum essential MVC services to the specified IServiceCollection. Additional services including MVC's
    support for authorization, formatters, and validation must be added separately using the IMvcCoreBuilder returned
    from this method.
    _config =

The MvcCoreServiceCollectionExtensions.AddMvcCore(IServiceCollection) approach for configuring MVC is provided
for experienced MVC developers who wish to have full control over the set of default services
// This method registered. MvcCoreServiceCollectionExtensions.AddMvcCore(IServiceCollection) will register the minimum set of
// For more information services necessary to route requests and invoke controllers. It is not expected that any application will satisfy its
0 references
public void ConfigureServices requirements with just a call to MvcCoreServiceCollectionExtensions.AddMvcCore(IServiceCollection). Additional
{
    configuration using the IMvcCoreBuilder will be required.
    services.AddMvcCore(option => option.EnableEndpointRouting = false);
}

```

Вижда се в подсказките, че AddMvcCore е с минимални възможности, а AddMvc – с пълни.

```

}
// This method
// For more information
0 references
public void ConfigureServices
{
    Returns:
    An IMvcBuilder that can be used to further configure the MVC services.
    services.AddMvc(option => option.EnableEndpointRouting = false);
}

```

Нека ги оставим на пълните възможности AddMvc.